

Week 3 – Working with *NumPy* and *Pandas*

Working with other Data Types

Vectors

A vector is a one-dimensional array of values, and you can perform vector-based operations, such as addition, subtraction, and inner product. The following script show how to perform vector-based operations.

```
v1 = [1,2,3]
v2 = [1,2,3]
v3 = [5,5,5]
s1 = [0,0,0]
d1 = [0,0,0]
p1 = 0

print("Initial Vectors")

print('v1:',v1)
print('v2:',v2)
print('v3:',v3)

for i in range(len(v1)):
    d1[i] = v3[i] - v2[i]
    s1[i] = v3[i] + v2[i]
    p1 = v3[i] * v2[i] + p1

print("After operations")

print('d1:',d1)
print('s1:',s1)
print('p1:',p1)
```

The previous example starts with the definition of three lists in Python, each of which represents a vector. The lists **d1** and **s1** represent the difference of **v2** and the **sum v2**, respectively. The number **p1** represents the “inner product” (also called the “dot product”) of **v3** and **v2**. The output is here:

Initial Vectors

v1: [1, 2, 3]

```
v2: [1, 2, 3]
```

```
v3: [5, 5, 5]
```

After operations

```
d1: [4, 3, 2]
```

```
s1: [6, 7, 8]
```

```
p1: 30
```

Matrices

A two-dimensional matrix is a two-dimensional array of values, and you can easily create such a matrix. For example, the following code block illustrates how to access different elements in a 2D matrix:

```
mm = [ ["a","b","c"], ["d","e","f"], ["g","h","i"] ];  
print 'mm: ',mm  
print 'mm[0]: ',mm[0]  
print 'mm[0][1]: ',mm[0][1]
```

The output from the preceding code block is here:

```
mm: [['a', 'b', 'c'], ['d', 'e', 'f'], ['g', 'h', 'i']]  
mm[0]: ['a', 'b', 'c']  
mm[0][1]: b
```

The following script illustrates how to create and populate a 2 two-dimensional matrix.

```
rows = 3  
cols = 3  
my2DMatrix = [[0 for i in range(rows)] for j in range(rows)]  
print('Before:',my2DMatrix)  
for row in range(rows):  
    for col in range(cols):  
        my2DMatrix[row][col] = row*row+col*col  
print('After: ',my2DMatrix)
```

The previous script initializes the variables **rows** and **cols** and then uses them to create the rows x cols matrix **my2DMatrix** whose values are initially 0. The next part contains a nested loop that

initializes the element of my2DMatrix whose position is **(row,col)** with the value **row*row+col*col**. The last line of code in prints the contents of my2DArray. The output is here:

Before: `[[0, 0, 0], [0, 0, 0], [0, 0, 0]]`

After: `[[0, 1, 4], [1, 2, 5], [4, 5, 8]]`

Using the NumPy Library for Matrices

The NumPy library (which you can install via pip) has a matrix object for manipulating matrices in Python. The following examples illustrate some of the features of NumPy. Initialize a matrix m and then display its contents:

```
>>> import numpy as np
>>> m = np.matrix([[1,-2,3],[0,4,5],[7,8,-9]])
>>> m
matrix([[ 1, -2,  3],
        [ 0,  4,  5],
        [ 7,  8, -9]])
```

The next snippet returns the transpose of matrix m:

```
>>> m.T
matrix([[ 1,  0,  7],
        [-2,  4,  8],
        [ 3,  5, -9]])
```

The next snippet returns the inverse of matrix m (if it exists):

```
>>> m.I
matrix([[ 0.33043478, -0.02608696,  0.09565217],
        [-0.15217391,  0.13043478,  0.02173913],
        [ 0.12173913,  0.09565217, -0.0173913 ]])
```

The next snippet defines a vector y and then computes the product **m*v**:

```
>>> v = np.matrix([[2],[3],[4]])
>>> v
matrix([[2],
        [3],
        [4]])
>>> m * v
```

```
matrix([[ 8],[32],[ 2]])
```

The next snippet imports the **numpy.linalg** subpackage and then computes the determinant of the matrix **m**:

```
>>> import numpy.linalg
```

```
>>> numpy.linalg.det(m)
```

```
-229.99999999999983
```

The next snippet finds the eigenvalues of the matrix **m**:

```
>>> numpy.linalg.eigvals(m)
```

```
array([-13.11474312,  2.75956154,  6.35518158])
```

The next snippet finds solutions to the equation **m*x = v**:

```
>>> x = numpy.linalg.solve(m, v)
```

```
>>> x
```

```
matrix([[ 0.96521739],  
[ 0.17391304],  
[ 0.46086957]])
```

In addition to the preceding samples, the NumPy package provides additional functionality, which you can find by performing an Internet search for articles and tutorials.

Queues

A queue is a FIFO (“First In First Out”) data structure. Thus, the oldest item in a queue is removed when a new item is added to a queue that is already full. Earlier we discussed how to use a Python List to emulate a queue. However, there is also a queue object in Python. The following code snippets illustrate how to use a queue in Python.

```
>>> from collections import deque
```

```
>>> q = deque('',maxlen=10)
```

```
>>> for i in range(10,20):
```

```
... q.append(i)
```

```
...
```

```
>>> print q
```

```
deque([10, 11, 12, 13, 14, 15, 16, 17, 18, 19], maxlen=10)
```

Tuples

Python supports a data type called a tuple that consists of comma-separated values without brackets (square brackets are for lists, round brackets are for arrays, and curly braces are for dictionaries). Various examples of Python tuples are here:

<https://docs.python.org/3.6/tutorial/datastructures.html#tuples-and-sequences>

The following code block illustrates how to create a tuple and create new tuples from an existing type in Python. Define a Python tuple `t` as follows:

```
>>> t = 1, 'a', 2, 'hello', 3
```

```
>>> t
```

```
(1, 'a', 2, 'hello', 3)
```

Display the first element of `t`:

```
>>> t[0]
```

```
1
```

Create a tuple `v` containing 10, 11, and `t`:

```
>>> v = 10, 11, t
```

```
>>> v
```

```
(10, 11, (1, 'a', 2, 'hello', 3))
```

Try modifying an element of `t` (which is immutable):

```
>>> t[0] = 1000
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
TypeError: 'tuple' object does not support item assignment
```

Python "deduplication" is useful because you can remove duplicates from a set and obtain a list, as shown here:

```
>>> lst = list(set(lst))
```

Note: The "in" operator on a list to search is **O(n)** whereas the "in" operator on set is **O(1)**.

Sets

A Python set in Python is an unordered collection that does not contain duplicate elements. Use curly braces or the **set()** function to create sets. Set objects support set-theoretic operations such as union, intersection, and difference.

Note: **set()** is required in order to create an empty set because {} creates an empty dictionary.

The following code block illustrates how to work with a Python set. Create a list of elements:

```
>>> l = ['a', 'b', 'a', 'c']
```

Create a set from the preceding list:

```
>>> s = set(l)
```

```
>>> s
```

```
set(['a', 'c', 'b'])
```

Test if an element is in the set:

```
>>> 'a' in s
```

```
True
```

```
>>> 'd' in s
```

```
False
```

```
>>>
```

Create a set from a string:

```
>>> n = set('abacad')
```

```
>>> n
```

```
set(['a', 'c', 'b', 'd'])
```

```
>>>
```

Subtract n from s:

```
>>> s - n
```

```
set([])
```

Subtract s from n:

```
>>> n - s
```

```
set(['d'])
```

```
>>>
```

The union of s and n:

```
>>> s | n
```

```
set(['a', 'c', 'b', 'd'])
```

The intersection of s and n:

```
>>> s & n
```

```
set(['a', 'c', 'b'])
```

The exclusive-or of s and n:

```
>>> s ^ n
```

```
set(['d'])
```

Dictionaries

Python has a key/value structure called a "**dict**" that is a hash table. A Python dictionary (and hash tables in general) can retrieve the value of a key in constant time, regardless of the number of entries in the dictionary (and the same is true for sets). You can think of a set as essentially just the keys (not the values) of a dict implementation. The contents of a dict can be written as a series of **key:value** pairs, as shown here:

```
dict1 = {key1:value1, key2:value2, ... }
```

The "empty dict" is just an empty pair of curly braces {}.

A Python dictionary (or hash table) contains of colon-separated key/ value bindings inside a pair of curly braces, as shown here:

```
dict1 = {}
```

```
dict1 = {'x' : 1, 'y' : 2}
```

The preceding code snippet defines dict1 as an empty dictionary, and then adds two key/value bindings. You can display the contents of dict1 with the following code:

```
>>> dict1 = {'x':1,'y':2}
```

```
>>> dict1
```

```
{'y': 2, 'x': 1}
```

```
>>> dict1['x']
```

```
1
```

```
>>> dict1['y']
```

```
2
```

```
>>> dict1['z']
```

```
Traceback (most recent call last):
```

```
File "<stdin>", line 1, in <module>
```

```
KeyError: 'z'
```

Note: Key/value bindings for a dict and a set are not necessarily stored in the same order that you defined them. Python dictionaries also provide the get method in order to retrieve key values:

```
>>> dict1.get('x')
```

```
1
```

```
>>> dict1.get('y')
```

```
2
```

```
>>> dict1.get('z')
```

As you can see, the Python get method returns None (which is displayed as an empty string) instead of an error when referencing a key that is not defined in a dictionary. You can also use dict comprehensions to create dictionaries from expressions, as shown here:

```
>>> {x: x**3 for x in (1, 2, 3)}
```

```
{1: 1, 2: 8, 3: 37}
```

You can easily check for the presence of a key in a Python dictionary as follows:

```
>>> 'x' in dict1
```

```
True
```

```
>>> 'z' in dict1
```

```
False
```

Use square brackets for finding or setting a value in a dictionary. For example, **dict['abc']** finds the value associated with the key 'abc'. You can use strings, numbers, and tuples work as key values, and you can use any type as the value. If you access a value that is not in the dict, Python throws a **KeyError**. Consequently, use the "in" operator to check if the key is in the dict. Alternatively, use **dict.get(key)** which returns the value or None if the key is not present. You can even use the expression **get(key, notfound-string)** to specify the value to return if a key is not found.

There are also other 7 sequence Types supported in Python: **str**, **unicode**, **list**, **tuple**, **bytearray**, **buffer**, and **xrange**. You can iterate through a sequence and retrieve the position index and corresponding value at the same time using the **enumerate()** function.

```
>>> for i, v in enumerate(['x', 'y', 'z']):
```

```
...     print i, v
```

```
...
```



```
0 x
1 y
2 z
```

Bytearray objects are created with the built-in function **bytearray()**. Although buffer objects are not directly supported by Python syntax, you can create them via the built-in **buffer()** function. Objects of type **xrange** are created with the **xrange()** function. An xrange object is similar to a buffer in the sense that there is no specific syntax to create them. Moreover, xrange objects do not support operations such as slicing, concatenation or repetition.

At this point you have seen all the Python types that you will encounter working with Machine Learning. Now, we are going to discuss mutable and immutable types in Python.

Mutable and Immutable Types in Python

Python represents its data as objects. Some of these objects (such as lists and dictionaries) are mutable, which means you can change their content without changing their identity. Objects such as integers, floats, strings and tuples are objects that cannot be changed. The key point to understand is the difference between changing the value versus assigning a new value to an object; you cannot change a string but you can assign it a different value. This detail can be verified by checking the id value of an object, as shown below.

```
s = "abc"

print('id #1:', id(s))

print('first char:', s[0])

try:
    s[0] = "o"
except:
    print('Cannot perform reassignment')

s = "xyz"

print('id #2:', id(s))

s += "uvw"

print('id #3:', id(s))
```

The output of Listing 3.x is here:

```
id #1: 4297972672
first char: a
```

Cannot perform reassignment

id #2: 4299809336

id #3: 4299777872

Thus, a Python type is immutable if its value cannot be changed (even though it's possible to assign a new value to such a type), otherwise a Python type is mutable. The Python immutable objects are of type bytes, complex, float, int, str, or tuple. On the other hand, dictionaries, lists, and sets are mutable. The key in a hash table must be an immutable type. Since strings are immutable in Python, you cannot insert a string in the "middle" of a given text string unless you construct a second string using concatenation. For example, suppose you have the string: **"this is a string"** and you want to create the following string: **"this is a longer string"**. The following Python code block illustrates how to perform this task:

```
text1 = "this is a string"  
text2 = text1[0:10] + "longer" + text1[9:]  
print 'text1:',text1  
print 'text2:',text2
```

The output of the preceding code block is here:

```
text1: this is a string  
text2: this is a longer string
```

The **type()** primitive returns the type of any object, including Python primitives, functions, and user-defined objects. The following code sample displays the type of an integer and a string:

```
var1 = 123  
var2 = 456.78  
print("type var1: ",type(var1))  
print("type var2: ",type(var2))
```

The output of the preceding code block is here:

```
type var1: <type 'int'>  
type var2: <type 'float'>
```

Now we're going to start the main part of this lab session, where I will introduce and provide you some examples of the most important packages used in Machine Learning. We will start with the NumPy package, followed by a quick introduction to Pandas and some of its useful features. The Pandas package for Python provides a rich and powerful set of APIs for managing datasets. These

APIs are very useful for machine learning and deep learning tasks that involve dynamically “slicing and dicing” subsets of datasets.

NumPy

NumPy is one of the fundamental packages for scientific computing in Python. It contains functionality for multidimensional arrays, high-level mathematical functions such as linear algebra operations and the Fourier transform, and pseudorandom number generators. NumPy is modelled after MatLab, with support for lists, arrays, and so forth. NumPy is easier to use than Matlab, and it's very common in TensorFlow code as well as Python code.

The NumPy package provides the **ndarray** object that encapsulates multidimensional arrays of homogeneous data types. Many ndarray operations are performed in compiled code in order to improve performance. Keep in mind the following important differences between NumPy arrays and the standard Python sequences:

- NumPy arrays have a fixed size, whereas Python lists can expand dynamically. Whenever you modify the size of a **ndarray**, a new array is created and the original array is deleted.
- NumPy arrays are homogeneous, which means that the elements in a NumPy array must all have the same data type. Except for NumPy arrays of objects, the elements in NumPy arrays of any other data type must have the same size in memory.
- NumPy arrays support more efficient execution (and require less code) of various types of operations on large numbers of data.
- Many scientific Python-based packages rely on NumPy arrays, and knowledge of NumPy arrays is becoming increasingly important.

Now that you have a general idea about NumPy, let's delve into some examples that illustrate how to work with NumPy arrays, which is the topic of the next section.

NumPy Arrays

An array is a set of consecutive memory locations used to store data. Each item in the array is called an element. The number of elements in an array is called the dimension of the array. A typical array declaration is shown here:

```
arr1 = np.array([1,2,3,4,5])
```

The preceding code snippet declares **arr1** as an array of five elements, which you can access via **arr1[0]** through **arr1[4]**. Notice that the first element has an index value of 0, the second element has an index value of 1, and so forth. Thus, if you declare an array of 100 elements, then the 100th element has index value of 99. Note: The first position in a NumPy array has index 0. NumPy treats arrays as vectors. Math operations are performed element-by-element. Remember the following

difference: “doubling” an array multiplies each element by 2, whereas “doubling” a list appends a list to itself. The following script illustrates some operations on a NumPy array.

```
import numpy as np
list1 = [1,2,3,4,5]
print(list1)
arr1 = np.array([1,2,3,4,5])
print(arr1)
list2 = [(1,2,3), (4,5,6)]
print(list2)
arr2 = np.array([(1,2,3), (4,5,6)])
print(arr2)
```

We define the variables **list1** and **list2** (which are Python lists), as well as the variables **arr1** and **arr2** (which are arrays) and print their values. The output from is here:

```
[1, 2, 3, 4, 5]
[1 2 3 4 5]
[(1, 2, 3), (4, 5, 6)]
[[1 2 3]
 [4 5 6]]
```

As you can see, Python lists and arrays are very easy to define, and now we’re ready to look at some loop operations for lists and arrays.

Regarding loops, the following script illustrates how to iterate through the elements of a NumPy array and a Python list.

```
import numpy as np
list = [1,2,3]
arr1 = np.array([1,2,3])
for e in list:
    print(e)
for e in arr1:
    print(e)
list1 = [1,2,3,4,5]
```

The previous script initializes the variable `list`, which is a Python list, and also the variable `arr1`, which is a NumPy array. The next portion contains two loops, each of which iterates through the elements in `list` and `arr1`. As you can see, the syntax is identical in both loops. The output is here:

```
1
2
3
1
2
3
```

The following script illustrates how to append elements to a NumPy array and a Python list.

```
import numpy as np
arr1 = np.array([1,2,3])
# these do not work:
#arr1.append(4)
#arr1 = arr1 + [5]
arr1 = np.append(arr1,4)
arr1 = np.append(arr1,[5])
for e in arr1:
    print(e)
arr2 = arr1 + arr1
for e in arr2:
    print(e)
```

The previous script initializes the variable `list`, which is a Python list, and also the variable `arr1`, which is a NumPy array. The output is:

```
1
2
3
4
5
2
4
```

6

8

10

The following script illustrates how to multiply elements in a Python list and a NumPy array.

```
import numpy as np
list1 = [1,2,3]
arr1 = np.array([1,2,3])
print('list: ',list1)
print('arr1: ',arr1)
print('2*list:',2*list)
print('2*arr1:',2*arr1)
```

The previous script contains a Python list called `list` and a NumPy array called `arr1`. The `print()` statements display the contents of `list` and `arr1` as well as the result of doubling `list1` and `arr1`. Recall that “doubling” a Python list is different from doubling a Python array, which you can see in the output from launching the previous code:

```
('list: ', [1, 2, 3])
('arr1: ', array([1, 2, 3]))
('2*list:', [1, 2, 3, 1, 2, 3])
('2*arr1:', array([2, 4, 6]))
```

The following script shows how to double the elements in a Python list.

```
import numpy as np
list1 = [1,2,3]
list2 = []
for e in list1:
    list2.append(2*e)
print('list1:',list1)
print('list2:',list2)
```

The script contains a Python list called `list1` and an empty NumPy list called `list2`. The next code snippet iterates through the elements of `list1` and appends them to the variable `list2`. The pair of `print()` statements display the contents of `list1` and `list2` to show you that they are the same. The output is here:

```
('list: ', [1, 2, 3])  
('list2:', [2, 4, 6])
```

The following script shows how to compute exponents of the elements in a Python list.

```
import numpy as np  
  
list1 = [1,2,3]  
list2 = []  
  
for e in list1:  
    list2.append(e*e) # e*e = squared  
  
print('list1:',list1)  
print('list2:',list2)
```

The previous code contains a Python list called **list1** and an empty NumPy list called **list2**. The next code snippet iterates through the elements of **list1** and appends the square of each element to the variable **list2**. The pair of **print()** statements display the contents of **list1** and **list2**. The output is here:

```
('list1:', [1, 2, 3])  
('list2:', [1, 4, 9])
```

Other Useful NumPy Methods

In addition to the NumPy methods that you saw in the code samples prior to this section, the following (often intuitively named) NumPy methods are also very useful.

- **np.zeros()** initializes an array with 0 values
- **np.ones()** initializes an array with 1 values
- **np.empty()** initializes an array with 0 values
- **np.arange()** provides a range of numbers
- **np.shape()** displays the shape of an object
- **np.reshape()** <= very useful!
- **np.linspace()** <= useful in regression
- **np.mean()** computes the mean of a set of numbers
- **np.std()** computes the standard deviation of a set of numbers

Although the **np.zeros()** and **np.empty()** both initialize a 2D array with 0, **np.zeros()** requires less execution time. You could also use **np.full(size, 0)**, but this method is the slowest of all three methods. The **reshape()** method and the **linspace()** method are very useful for changing the dimensions of an array and generating a list of numeric values, respectively. The **reshape()** method often appears in TensorFlow code, and the **linspace()** method is useful for generating a set of

numbers in linear regression (that we will discuss next week). The **mean()** and **std()** methods are useful for calculating the mean and the standard deviation of a set of numbers. For example, you can use these two methods in order to resize the values in a Gaussian distribution so that their mean is 0 and the standard deviation is 1. This process is called standardizing a Gaussian distribution.

NumPy arrays support the “reshape” method that enables you to restructure the dimensions of an array of numbers. In general, if a NumPy array contains m elements, where m is a positive integer, then that array can be restructured as an $m_1 \times m_2$ NumPy array, where m_1 and m_2 are positive integers such that $m_1 * m_2 = m$. The following script illustrates how to use the **reshape()** method on a NumPy array.

```
import numpy as np

x = np.array([[2, 3], [4, 5], [6, 7]])

print(x.shape) # (3, 2)

x = x.reshape((2, 3))

print(x.shape) # (2, 3)

print('x1:', x)

x = x.reshape((_1))

print(x.shape) # (6,)

print('x2:', x)

x = x.reshape((6, _1))

print(x.shape) # (6, 1)

print('x3:', x)

x = x.reshape((_1, 6))

print(x.shape) # (1, 6)

print('x4:', x)
```

The previous code contains a NumPy array called `x` whose dimensions are 3x2, followed by a set of invocations of the **reshape()** method that reshape the contents of `x`. The first invocation of the **reshape()** method changes the shape of `x` from 3x2 to 2x3. The second invocation changes the shape of `x` from 2x3 to 6x1. The third invocation changes the shape of `x` from 1x6 to 6x1. The final invocation changes the shape of `x` from 6x1 to 1x6 again. Each invocation of the **reshape()** method is followed by a `print()` statement so that you can see the effect of the invocation. The output is:

```
(3, 2)

(2, 3)

('x1:', array([[2, 3, 4],
```



```
[5, 6, 7]])
(6,)
('x2:', array([2, 3, 4, 5, 6, 7]))
(6, 1)
('x3:', array([
[3],
[4],
[5],
[6],
[7]]))
(1, 6)
```

Calculating the Mean and Standard Deviation – Example 1

NumPy provides various **built-in functions** that perform statistical calculations, such as the following list of methods:

```
np.linspace() <= useful for regression
np.mean()
np.std()
```

The **np.linspace()** method generates a set of equally spaced numbers between a lower bound and an upper bound. The **np.mean()** and **np.std()** methods calculate the mean and standard deviation, respectively, of a set of numbers. The following script illustrates how to calculate statistical values from a NumPy array.

```
import numpy as np
x2 = np.arange(8)
print 'mean = ',x2.mean()
print 'std = ',x2.std()
x3 = (x2 - x2.mean())/x2.std()
print 'x3 mean = ',x3.mean()
print 'x3 std = ',x3.std()
```

The script contains a NumPy array **x2** that consists of the first eight integers. Next, the **mean()** and **std()** that are “associated” with **x2** are invoked in order to calculate the mean and standard deviation, respectively, of the elements of **x2**. The output is here:

```
('a: ', array([1, 2]))  
('b: ', array([3, 4]))
```

Calculating the Mean and Standard Deviation – Example 2

The code sample in this section extends the code sample in the previous section with additional statistical values, and the code below can be used for any data distribution. Keep in mind that the code sample uses random numbers simply for the purposes of illustration: after you have launched the code sample, replace those numbers with values from a CSV file or some other dataset containing meaningful values. Moreover, this section does not provide details regarding the meaning of quartiles, but you can learn about quartiles here: <https://en.wikipedia.org/wiki/Quartile>

The following script illustrates how to display various statistical values from a NumPy array of random numbers.

```
import numpy as np  
  
from numpy import percentile  
from numpy.random import rand  
  
# generate data sample  
data = np.random.rand(1000)  
  
# calculate quartiles, min, and max  
quartiles = percentile(data, [25, 50, 75])  
data_min, data_max = data.min(), data.max()  
  
# print summary information  
print('Minimum: %.3f' % data_min)  
print('Q1 value: %.3f' % quartiles[0])  
print('Median: %.3f' % quartiles[1])  
print('Mean Val: %.3f' % data.mean())  
print('Std Dev: %.3f' % data.std())  
print('Q3 value: %.3f' % quartiles[2])  
print('Maximum: %.3f' % data_max)
```

The data sample (shown in bold) is from a uniform distribution between 0 and 1. The NumPy **percentile()** function calculates a linear interpolation (average) between observations, which is needed to calculate the median on a sample with an even number of values. As you can surmise, the NumPy functions **min()** and **max()** calculate the smallest and largest values in the data sample. The output is here:

Minimum: 0.000

Q1 value: 0.237

Median: 0.500

Mean Val: 0.495

Std Dev: 0.295

Q3 value: 0.747

Maximum: 0.999

Pandas

Pandas is a Python package that is compatible with other Python packages, such as NumPy, Matplotlib, and so forth. In many ways the Pandas package has the semantics of a spreadsheet, and it also works with xls, xml, html, csv file types. Pandas provides a data type called a **DataFrame** (similar to a Python dictionary) with extremely powerful functionality, which is discussed in the next section. Pandas DataFrames support a variety of input types, such as ndarrays, lists, dicts, or Series. Pandas also provides another data type called Pandas Series (not discussed in this workbook), this data structure provides another mechanism for managing data (search online for more details).

Pandas Dataframes

In simplified terms, a Pandas DataFrame is a two-dimensional data structure, and it is convenient to think of the data structure in terms of rows and columns. DataFrames can be labelled (rows as well as columns), and the columns can contain different data types. By way of analogy, it might be useful to think of a DataFrame as the counterpart to a spreadsheet, which makes it a very useful data type in Pandas-related Python scripts. The source of the dataset can be a data file, database tables, Web service, and so forth. Pandas DataFrame features include:

- DataFrame methods
- DataFrame statistics
- Grouping, pivoting, and reshaping
- Dealing with missing data
- Joining dataframes

Dataframes and Data Cleaning Tasks

The specific tasks that you need to perform depend on the structure and contents of a dataset. In general, you will perform a workflow with the following steps (not necessarily always in this order), all of which can be performed with a Pandas DataFrame: read data into a dataframe; display top of dataframe; display column data types; display non_missing values; replace NA with a value; iterate through the columns; statistics for each column; find missing values; total missing values; percentage of missing values; sort table values; print summary information; columns with > 50% missing; rename columns.

A Labelled Pandas Dataframe

The following script illustrates how to define a Pandas DataFrame whose rows and columns are labelled.

```
import numpy
import pandas

myarray = numpy.array([[10,30,20],
                       [50,40,60], [1000,2000,3000]])

rownames = ['apples', 'oranges', 'beer']
colnames = ['January', 'February', 'March']

mydf = pandas.DataFrame(myarray, index=rownames, columns=colnames)

print(mydf)

print(mydf.describe())
```

The previous code contains two important statements followed by the variable **myarray**, which is a 3x3 NumPy array of numbers. The variables **rownames** and **colnames** provide names for the rows and columns, respectively, of the data in **myarray**. Next, the variable **mydf** is initialized as a Pandas DataFrame with the specified datasource (i.e., **myarray**). You might be surprised to see that the first portion of the following output requires a single print statement (which simply displays the contents of **mydf**). The second portion of the output is generated by invoking the **describe()** method that is available for any NumPy DataFrame. The **describe()** method is very useful: you will see various statistical quantities (e.g., mean, standard deviation minimum, and maximum) performed column_wise (not row_wise), along with values for the 25th, 50th, and 75th percentiles. The output is here:

	January	February	March
apples	10	30	20
oranges	50	40	60
beer	1000	2000	3000
	January	February	March
count	3.000000	3.000000	3.000000
mean	353.333333	690.000000	1026.666667
std	560.386771	1134.504297	1709.073823
min	10.000000	30.000000	20.000000
25%	30.000000	35.000000	40.000000
50%	50.000000	40.000000	60.000000
75%	525.000000	1020.000000	1530.000000
max	1000.000000	2000.000000	3000.000000

Pandas Numeric DataFrames

The following script illustrates how to define a Pandas DataFrame whose rows and columns are numbers (but the column labels are characters).

```
import numpy as np

import pandas as pd

df1 = pd.DataFrame(np.random.randn(10,4), columns=['A', 'B', 'C', 'D'])
df2 = pd.DataFrame(np.random.randn(7, 3), columns=['A', 'B', 'C'])
df3 = df1 + df2
```

The essence of the previous script is initializing the DataFrames df1 and df2, and then defining the DataFrame df3 as the sum of df1 and df2. The output is here:

	A	B	C	D
0	1.938212	1.117058	-0.013852	NaN
1	-1.097118	-2.593989	-1.889432	NaN
2	-2.288094	1.739121	-1.136556	NaN
3	2.183231	0.766987	1.246965	NaN
4	0.567165	1.244130	-0.486730	NaN
5	-2.633420	-1.130725	0.918430	NaN
6	-0.907690	-2.429952	-0.267623	NaN
7	NaN	NaN	NaN	NaN
8	NaN	NaN	NaN	NaN
9	NaN	NaN	NaN	NaN

Keep in mind that the default behaviour for operations involving a DataFrame and Series is to align the Series index on the DataFrame columns; this results in a row-wise output. Here is a simple illustration:

```
import pandas as pd

names = pd.Series(['SF', 'San Jose', 'Sacramento'])
sizes = pd.Series([852469, 1015785, 485199])

df = pd.DataFrame({ 'Cities': names, 'Size': sizes })
df = pd.DataFrame({ 'City name': names, 'sizes': sizes })
```

```
print(df)
```

The output of the preceding code block is here:

	City name	sizes
0	SF	852469
1	San Jose	1015785
2	Sacramento	485199

Transposing a Pandas Dataframe

The T attribute (as well as the transpose function) enables you to generate the transpose of a Pandas DataFrame, similar to a NumPy ndarray. For example, the following code snippet defines a Pandas DataFrame df1 and then displays the transpose of df1:

```
import pandas as pd

df1 = pd.DataFrame({'a' : [1, 0, 1], 'b' : [0, 1, 1] }, dtype=int)

print("df1.T:")

print(df1.T)
```

The output is here:

```
df1.T:
   0  1  2
a  1  0  1
b  0  1  1
```

The following code snippet defines Pandas dataFrames df1 and df2 and then displays their sum:

```
import pandas as pd

df1 = pd.DataFrame({'a' : [1, 0, 1], 'b' : [0, 1, 1] }, dtype=int)
df2 = pd.DataFrame({'a' : [3, 3, 3], 'b' : [5, 5, 5] }, dtype=int)

print("df1 + df2:")

print(df1 + df2)
```

The output is here:

```
df1.T:
   0  1  2
a  1  0  1
b  0  1  1
df1 + df2:
   a  b
0  4  5
1  3  6
2  4  6
```

Pandas Dataframes and Random Numbers

The following script illustrates how to create a Pandas DataFrame with random numbers.

```
import pandas as pd

df = pd.DataFrame(np.random.randint(1, 5, size=(5, 2)),
                  columns=['a', 'b'])

df = df.append(df.agg(['sum', 'mean']))

print("Contents of dataframe:")

print(df)
```

The previous script defines the Pandas DataFrame `df` that consists of 5 rows and 2 columns of random integers between 1 and 5. Notice that the columns of `df` are labelled “a” and “b.” In addition, the next code snippet appends two rows consisting of the sum and the mean of the numbers in both columns. The output is here:

```
df1.T:
   0  1  2
a  1  0  1
b  0  1  1
Contents of dataframe:
      a  b
0     1.0 1.0
1     4.0 4.0
2     2.0 3.0
3     2.0 1.0
4     3.0 3.0
sum    12.0 12.0
mean     2.4 2.4
```

Combining Pandas DataFrames

The following script illustrates how to combine Pandas DataFrames.

```
import pandas as pd

import numpy as np

df = pd.DataFrame({'foo1' : np.random.randn(5),
                  'foo2' : np.random.randn(5)})

print("contents of df:")

print(df)

print("contents of foo1:")

print(df.foo1)

print("contents of foo2:")

print(df.foo2)
```

The script defines the Pandas DataFrame `df` that consists of 5 rows and 2 columns (labelled “**foo1**” and “**foo2**”) of random real numbers between 0 and 5. The next portion of the script displays the contents of `df` and **foo1**. The output is here:

```
contents of df:
   foo1    foo2
0 -0.849124 -0.039625
1 -1.555710  0.099625
2 -0.539104  1.396591
3 -0.315146 -0.932003
4  0.135923 -1.049802
contents of foo1:
0    -0.849124
1    -1.555710
2    -0.539104
3    -0.315146
4     0.135923
Name: foo1, dtype: float64
contents of foo2:
0    -0.039625
1     0.099625
2     1.396591
3    -0.932003
4    -1.049802
Name: foo2, dtype: float64
```

Pandas supports the “**concat**” method in DataFrames in order to concatenate DataFrames. The following script illustrates how to combine two Pandas DataFrames by using **concat**.

```
import pandas as pd

can_weather = pd.DataFrame({
    "city": ["Vancouver", "Toronto", "Montreal"],
    "temperature": [72, 65, 50],
    "humidity": [40, 20, 25]
})

us_weather = pd.DataFrame({
    "city": ["SF", "Chicago", "LA"],
    "temperature": [60, 40, 85],
    "humidity": [30, 15, 55]
})

df = pd.concat([can_weather, us_weather])

print(df)
```

The first line in the script is an import statement, followed by the definition of the Pandas dataframes `can_weather` and `us_weather` that contain weather-related information for cities in Canada and the United States, respectively. The Pandas dataframe `df` is the concatenation of `can_weather` and `us_weather`. The output from is here:

	city	temperature	humidity
0	Vancouver	72	40
1	Toronto	65	20
2	Montreal	50	25
0	SF	60	30
1	Chicago	40	15
2	LA	85	55

Data Manipulation with Pandas Dataframes – Example 1

As a simple example, suppose that we have a two-person company that keeps track of income and expenses on a quarterly basis, and we want to calculate the profit/loss for each quarter, and also the overall profit/loss. The following script illustrates how to define a Pandas DataFrame consisting of income-related values.

```
import pandas as pd

summary = {
    'Quarter': ['Q1', 'Q2', 'Q3', 'Q4'],
    'Cost': [23500, 34000, 57000, 32000],
    'Revenue': [40000, 40000, 40000, 40000]
}

df = pd.DataFrame(summary)

print("Entire Dataset:\n",df)
print("Quarter:\n",df.Quarter)
print("Cost:\n",df.Cost)
print("Revenue:\n",df.Revenue)
```

The script defines the variable `summary` that contains hard-coded quarterly information about cost and revenue for our two-person company. In general, these hard-coded values would be replaced by data from another source (such as a CSV file), so think of this code sample as a simple way to illustrate some of the functionality that is available in Pandas DataFrames. The variable `df` is a Pandas DataFrame based on the data in the `summary` variable. The three `print` statements display the quarters, the cost per quarter, and the revenue per quarter. The output is here:

```

Entire Dataset:
  Quarter  Cost  Revenue
0      Q1  23500   40000
1      Q2  34000   40000
2      Q3  57000   40000
3      Q4  32000   40000
Quarter:
0      Q1
1      Q2
2      Q3
3      Q4
Name: Quarter, dtype: object
Cost:
0      23500
1      34000
2      57000
3      32000
Name: Cost, dtype: int64
Revenue:
0      40000
1      40000
2      40000
3      40000
Name: Revenue, dtype: int64

```

Data Manipulation with Pandas DataFrames - Example 2

In this section, let's suppose that we have a two-person company that keeps track of income and expenses on a quarterly basis, and we want to calculate the profit/loss for each quarter, and also the overall profit/loss. The following script illustrates how to define a Pandas DataFrame consisting of income-related values.

```

import pandas as pd

summary = {
    'Quarter': ['Q1', 'Q2', 'Q3', 'Q4'],
    'Cost': [23500, 34000, 57000, 32000],
    'Revenue': [40000, 40000, 40000, 40000]
}

df = pd.DataFrame(summary)

print("First Dataset:\n",df)

df['Total'] = df.sum(axis=1)

print("Second Dataset:\n",df)

```

The previous script defines the variable `summary` that contains quarterly information about cost and revenue for our two-person company. The variable `df` is a Pandas DataFrame based on the data in the `summary` variable. The three print statements display the quarters, the cost per quarter, and the revenue per quarter. The output is here:

First Dataset:				
	Quarter	Cost	Revenue	
0	Q1	23500	40000	
1	Q2	34000	40000	
2	Q3	57000	40000	
3	Q4	32000	40000	
Second Dataset:				
	Quarter	Cost	Revenue	Total
0	Q1	23500	40000	63500
1	Q2	34000	40000	74000
2	Q3	57000	40000	97000
3	Q4	32000	40000	72000

Data Manipulation with Pandas Dataframes – Example 3

Let's start with the same assumption as the previous section: we have a two-person company that keeps track of income and expenses on a quarterly basis, and we want to calculate the profit/loss for each quarter, and also the overall profit/loss. In addition, we want to compute column totals and row totals. The following script illustrates how to define a Pandas DataFrame consisting of income-related values.

```
import pandas as pd

summary = {
    'Quarter': ['Q1', 'Q2', 'Q3', 'Q4'],
    'Cost': [23500, 34000, 57000, 32000],
    'Revenue': [40000, 40000, 40000, 40000]
}

df = pd.DataFrame(summary)
print("First Dataset:\n",df)
df['Total'] = df.sum(axis=1)
df.loc['Sum'] = df.sum()
print("Second Dataset:\n",df)
# or df.loc['avg'] / 3
#df.loc['avg'] = df[:3].mean()
#print("Third Dataset:\n",df)
```

The previous script defines the variable `summary` that contains quarterly information about cost and revenue for our two-person company. The variable `df` is a Pandas DataFrame based on the data in the `summary` variable. The three print statements display the quarters, the cost per quarter, and the revenue per quarter. The output is here:

First Dataset:				
	Quarter	Cost	Revenue	
0	Q1	23500	40000	
1	Q2	34000	40000	
2	Q3	57000	40000	
3	Q4	32000	40000	
Second Dataset:				
	Quarter	Cost	Revenue	Total
0	Q1	23500	40000	63500
1	Q2	34000	40000	74000
2	Q3	57000	40000	97000
3	Q4	32000	40000	72000
Sum	Q1Q2Q3Q4	146500	160000	306500

Pandas DataFrames and CSV Files

The code samples in several earlier sections contain hard-coded data inside the Python scripts. However, it's also very common to read data from a CSV file. You can use the Python `csv.reader()` function, the NumPy `loadtxt()` function, or the Pandas function `read_csv()` function (shown in this section) to read the contents of CSV files. The following script illustrates how to read a CSV file, initialize a Pandas DataFrame with the contents of that CSV file, and display various subsets of the data in the Pandas DataFrames. The CSV file used in this example consists of the weather forecasts of Middlesbrough that you can download from <https://www.visualcrossing.com/> after registering and the following explains how to download data (dataset has been included in the Learning material of this week): <https://www.visualcrossing.com/resources/documentation/weather-data-tutorials/how-can-i-download-weather-forecast-data-as-a-csv-file/>

```
import pandas as pd

df = pd.read_csv("forecast_data.csv")

print(df)

print(df.shape) # rows, columns

print(df.head()) # df.head(3)

print(df.tail())

print(df[1:3])

print(df.columns)

print(df['Temperature'])

print(df['Chance Precipitation (%)'])

print(df['Relative Humidity'])

print(df['Conditions'])

print(df['Wind Speed'])
```

The previous script invokes the Pandas **read_csv()** function to read the contents of the CSV file `forecast_data.csv`, followed by a set of Python **print()** statements that display various portions of the CSV file.

In some situations, you might need to apply Boolean conditional logic to “filter out” some rows of data, based on a conditional condition that is applied to a column value. The following script illustrates how to manipulate the data of the previous dataframe.

```
import pandas as pd

# initialize list of lists
data = [['Tom', 20], ['Paul', 35], ['Julie', 14], ['James', 28],
        ['Myriam', 22], ['Kate', 27]]

# Create the pandas DataFrame
df = pd.DataFrame(data, columns = ['Name', 'Age'])

df.info()

print('Name:')
print(df['Name'])
print('_____')
print('age over 24:')
print(df['age'] > 24)
print('_____')
print('age over 24:')
myfilter = df['age'] > 24
print(df[myfilter])
```

The previous script creates Pandas dataframe `df` with the contents of the list we defined. The next portion displays the structure of `df`, followed by the first names of all the people. The next portion displays a tabular list of six rows containing either True or False depending on whether a person is over 24 or at most 24, respectively. The final portion of the script displays a tabular list of two rows containing all the details of the people who are over 24. The output is here:

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 2 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Name    6 non-null      object
1    Age      6 non-null      int64
dtypes: int64(1), object(1)
memory usage: 224.0+ bytes
Name:
0      Tom
1      Paul
2      Julie
3      James
4      Myriam
5      Kate
Name: Name, dtype: object

Age over 24:
0    False
1     True
2    False
3     True
4    False
5     True
Name: Age, dtype: bool

Age over 24:
   Name  Age
1  Paul   35
3 James   28
5  Kate   27

```

Pandas DataFrames and Excel Spreadsheets

The following script illustrates how to read data from an Excel spreadsheet and create a Pandas DataFrame with that data.

```

import pandas as pd

df = pd.read_excel("students_machine_learning.xlsx")

pd.set_option("display.max_rows", None, "display.max_columns",
None)

print("Contents of Excel spreadsheet:")

print(df)

```

The script is straightforward: the Pandas dataframe `df` is initialized with the contents of the spreadsheet “students_machine_learning.xlsx” via the Pandas function `read_excel()`. The output is the list of students of this module.

Select, Add, and Delete Columns in DataFrames

This section contains short code blocks that illustrate how to perform operations on a DataFrame that resemble the operations on a Python dictionary. For example, getting, setting, and deleting columns works with the same syntax as the analogous Python dict operations, as shown here:

```
df = pd.DataFrame.from_dict(dict([( 'A' , [1,2,3]), ( 'B' , [4,5,6]) ]),
orient = 'index', columns=[ 'one' , 'two' , 'three' ])
print(df)
```

The output from the preceding code snippet is here:

	one	two	three
A	1	2	3
B	4	5	6

Now look at the following sequence of operations on the contents of the dataframe **df**:

```
df['three'] = df['one'] * df['two']
df['flag'] = df['one'] > 2
print(df)
```

The output from the preceding code block is here:

	one	two	three	flag
A	1	2	2	False
B	4	5	20	True

Columns can be deleted or popped like with a Python dict, as shown in following code snippet:

```
del df['two']
three = df.pop('three')
print(df)
```

The output from the preceding code block is here:

	one	flag
A	1	False
B	4	True

When inserting a scalar value, it will naturally be propagated to fill the column:

```
df['foo'] = 'bar'
print(df)
```

The output from the preceding code snippet is here:

	one	flag	foo
A	1	False	bar
B	4	True	bar

When inserting a Series that does not have the same index as the DataFrame, it will be “conformed” to the index of the DataFrame:

```
df['one_trunc'] = df['one'][:2]
print(df)
```

The output from the preceding code snippet is here:

	one	two	three	one_trunc
A	1	2	3	1
B	4	5	6	4

You can insert raw ndarrays but their length must match the length of the index of the DataFrame.

Pandas DataFrames and Scatterplots

The following script illustrates how to generate a scatterplot from a Pandas DataFrame.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pandas import read_csv
from pandas.plotting import scatter_matrix
myarray = np.array([[10,30,20],[50,40,60],[1000,2000,3000]])
rownames = ['apples', 'oranges', 'beer']
colnames = ['January', 'February', 'March']
mydf = pd.DataFrame(myarray, index=rownames, columns=colnames)
print(mydf)
print(mydf.describe())
scatter_matrix(mydf)
plt.show()
```

The script starts with various import statements, followed by the definition of the NumPy array **myarray**. Next, the variables **myarray** and **colnames** are initialized with values for the rows and columns, respectively. The next portion initializes the Pandas DataFrame **mydf**, so that the rows and columns are labelled in the output, as shown here:

	January	February	March
apples	10	30	20
oranges	50	40	60
beer	1000	2000	3000
	January	February	March
count	3.000000	3.000000	3.000000
mean	353.333333	690.000000	1026.666667
std	560.386771	1134.504297	1709.073823
min	10.000000	30.000000	20.000000
25%	30.000000	35.000000	40.000000
50%	50.000000	40.000000	60.000000
75%	525.000000	1020.000000	1530.000000
max	1000.000000	2000.000000	3000.000000

Pandas DataFrames and Simple Statistics

The following script illustrates how to gather basic statistics from data in a Pandas DataFrame.

```
import pandas as pd

df = pd.read_csv("Housing.csv")

minimum_bdrms = df["bedrooms"].min()
median_bdrms = df["bedrooms"].median()
maximum_bdrms = df["bedrooms"].max()

print("minimum # of bedrooms:", minimum_bdrms)
print("median # of bedrooms:", median_bdrms)
print("maximum # of bedrooms:", maximum_bdrms)

print("")

print("median values:", df.median().values)

print("")

prices = df["price"]
print("first 5 prices:")
print(prices.head())

print("")

median_price = df["price"].median()
print("median price:", median_price)

print("")

corr_matrix = df.corr()
print("correlation matrix:")
print(corr_matrix["price"].sort_values(ascending=False))
```

The script initializes the Pandas DataFrame `df` with the contents of the CSV file `Housing.csv` (included in the learning material of this week). The next three variables are initialized with the minimum, median, and maximum number of bedrooms, respectively, and then these values are displayed. The next portion of the script initializes the variable `prices` with the contents of the `prices` column of the Pandas DataFrame `df`. Next, the first five rows are printed via the `prices.head()` statement, followed by the median value of the prices. The final portion of the script initializes the variable `corr_matrix` with the contents of the correlation matrix for the Pandas DataFrame `df`, and then displays its contents. The output is:

```
minimum # of bedrooms: 1
median # of bedrooms: 3.0
maximum # of bedrooms: 6

median values: [4.34e+06 4.60e+03 3.00e+00 1.00e+00 2.00e+00 0.00e+00]

first 5 prices:
0    13300000
1    12250000
2    12250000
3    12215000
4    11410000
Name: price, dtype: int64

median price: 4340000.0

correlation matrix:
price      1.000000
area       0.535997
bathrooms  0.517545
stories    0.420712
parking     0.384394
bedrooms   0.366494
Name: price, dtype: float64
```

Useful One_line Commands in Pandas

This section contains an eclectic mix of one-line commands in Pandas (some of which you have already seen in this workbook) that are useful to know:

Save a dataframe to a CSV file (comma separated and without indices):
`df.to_csv("data.csv", sep=",", index=False)`

List the column names of a DataFrame: `df.columns`

Drop missing data from a DataFrame: `df.dropna(axis=0, how='any')`

Replace missing data in a DataFrame: `df.replace(to_replace=None, value=None)`

Check for NANS in a DataFrame: `pd.isnull(object)`

Drop a feature in a DataFrame: `df.drop('feature_variable_name', axis=1)`

Convert object type to float in a DataFrame: `pd.to_numeric(df["feature_name"], errors='coerce')`

Convert data in a DataFrame to NumPy array: `df.as_matrix()`

Display the first n rows of a dataframe: `df.head(n)`

Get data by feature name in a DataFrame: `df.loc[feature_name]`

Apply a function to a DataFrame: multiply all values in the "height" column of the dataframe by 3:

`df["height"].apply(lambda height: 3 * height)` OR:

```
def multiply(x):
```

```
    return x * 3
```

```
df["height"].apply(multiply)
```

Rename the fourth column of the dataframe as "height": `df.rename(columns = {df.columns[3]: 'height'}, inplace=True)`

Get the unique entries of the column "first" in a DataFrame: `df["first"].unique()`

Create a dataframe with columns "first" and "last" from an existing DataFrame: `new_df = df[["name", "size"]]`

Sort the data in a DataFrame: `df.sort_values(ascending = False)`

Filter the data column named "size" to display only values equal to 7: `df[df["size"] == 7]`

Select the first row of the "height" column in a DataFrame: `df.loc([0], ['height'])`